

Multi-Threaded Data Aggregator

Full Working Solution & Explanation
BITE 2109 Capstone Project

Generated with tested, working code — every example in this document was run before inclusion.

A Note Before You Start

This assignment is significantly more advanced than dictionaries. It expects you to already know: HTTP requests, JSON, classes/OOP, `try/except` error handling, web scraping (BeautifulSoup), threading, `asyncio`, decorators (`@lru_cache`), functional tools (`map / filter / reduce`), profiling, and unit testing with mocks.

None of that is a criticism of you — it just means whoever assigned this either expects you to learn a lot of new material along the way, or this was meant for later in the course. Read this document as a **map of what's ahead**, not something you need to fully understand today. Sections you're not ready for yet are marked so you know what to revisit.

This document contains:

1. A working, tested solution to every part of the assignment
2. Line-by-line explanations of *why* the code is written the way it is
3. Concepts flagged by difficulty, so you know what's next after dictionaries
4. Full setup instructions for **Windows** and **Ubuntu**
5. Sample output and how to interpret it

All code in this document was actually run and tested before being included here.

1. Project Overview

The assignment asks you to build a command-line tool that:

1. **Fetches** product data from an API
2. **Scrapes** review data from a website
3. **Combines and analyzes** that data
4. **Outputs** a report (console + JSON)
5. Does all of this using **two different concurrency strategies** so you can compare their speed
6. Is covered by **unit tests**

Since the assignment allows “dummy/fake URLs... or simple mock servers,” this solution includes a small local mock server (`mock_server.py`) that pretends to be both the product API and the review website. This means the whole project runs on your own computer with no real external API needed.

Project structure

```
data_aggregator/
├── config.py           # constants (URLs, timeouts, filenames)
├── mock_server.py      # fake API + review site (run this first)
├── api_fetcher.py      # talks to the product API
├── scraper.py          # scrapes review pages with BeautifulSoup
├── data_models.py      # the Product class (__slots__)
├── analyzer.py         # map/filter/reduce data analysis
├── reporter.py         # console + JSON output
├── main.py            # ties it all together, both concurrency modes
├── test_project.py     # unit tests (pytest)
└── report.json         # generated after you run main.py
```

[Beginner note] Splitting a program into files like this is called **modularization**. Each file has ONE job. This isn't required for tiny scripts, but becomes essential once a project grows — it's much easier to find and fix a bug in a 60-line `scraper.py` than in one 600-line file.

2. Setup Instructions

2.1 Requirements (both OSes)

- Python 3.9 or newer
- pip (comes with Python)

- The following packages: `flask`, `requests`, `beautifulsoup4`, `aiohttp`, `pytest`

2.2 Windows Setup

1. **Install Python** (skip if already installed):

- Download from python.org/downloads
- **Important:** during install, check the box “**Add Python to PATH**”
- Verify in Command Prompt:

```
python --version
```

2. **Create a project folder and open it in Command Prompt or PowerShell:**

```
mkdir data_aggregator  
cd data_aggregator
```

Place all the `.py` files from this document into that folder.

3. **(Recommended) Create a virtual environment.** This keeps this project’s packages separate from other Python projects on your machine:

```
python -m venv venv  
venv\Scripts\activate
```

Your prompt should now show `(venv)` at the start of the line.

4. **Install dependencies:**

```
pip install flask requests beautifulsoup4 aiohttp pytest
```

5. **Run the mock server** (leave this window open/running):

```
python mock_server.py
```

6. **Open a SECOND Command Prompt window**, activate the venv again (`venv\Scripts\activate`), navigate to the same folder, and run the app:

```
python main.py --mode threads  
python main.py --mode async  
python main.py --mode threads --profile
```

7. **Run the tests:**

```
pytest test_project.py -v
```

2.3 Ubuntu Setup

1. **Install Python and pip** (usually pre-installed on Ubuntu; this ensures venv support too):

```
sudo apt update  
sudo apt install python3 python3-pip python3-venv  
python3 --version
```

2. **Create a project folder:**

```
mkdir data_aggregator  
cd data_aggregator
```

Place all the `.py` files from this document into that folder.

3. **Create and activate a virtual environment:**

```
python3 -m venv venv  
source venv/bin/activate
```

4. Install dependencies:

```
pip install flask requests beautifulsoup4 aiohttp pytest
```

5. Run the mock server (leave this terminal running):

```
python3 mock_server.py
```

6. Open a **SECOND** terminal tab/window, activate the venv again (`source venv/bin/activate`), `cd` into the folder, and run:

```
python3 main.py --mode threads
python3 main.py --mode async
python3 main.py --mode threads --profile
```

7. Run the tests:

```
pytest test_project.py -v
```

[Beginner note] A **virtual environment (venv)** is an isolated copy of Python just for one project, so installing packages here doesn't affect other projects or your system Python. You'll see this in almost every real Python project.

3. `config.py` — Configuration

[Beginner-friendly — you already know this] This is just variables. Centralizing settings like URLs in one file is good practice you can use immediately, even in your dictionary exercises.

```
"""
config.py
-----
Central place for constants. Keeping URLs and settings here (instead of
scattered through the code) means if the API address changes, you only
edit it in ONE place.

NOTE: These are dummy/mock URLs. Replace them with real ones, or point
them at the mock server included in this project (see mock_server.py).
"""

# Base URL of the product API (mocked locally by mock_server.py)
API_BASE_URL = "http://127.0.0.1:5000/api/products"

# Base URL of the "review site" we scrape (also mocked locally)
SCRAPER_BASE_URL = "http://127.0.0.1:5000/reviews"

# How long (seconds) we wait for a network response before giving up
REQUEST_TIMEOUT = 5

# Where the final report gets written
REPORT_OUTPUT_FILE = "report.json"
```

4. `mock_server.py` — The Fake API & Review Site

[Intermediate] This uses **Flask**, a web framework, to fake a real API and website locally. You don't need to understand Flask deeply — just know that `@app.route(...)` says "when someone visits this URL, run this function."

```
"""
mock_server.py
-----
The assignment says to use "dummy/fake URLs ... or create simple mock
servers". Since we don't have a real market-research API to call, this
file IS that mock server. It runs locally on your machine and pretends
to be both:

1. The product API      -> GET /api/products
                        GET /api/products/<id>
2. The review website  -> GET /reviews/<id> (returns HTML to scrape)

Run this in its own terminal window BEFORE running main.py:
```

```

python mock_server.py

It will keep running (serving requests) until you press Ctrl+C.
"""

from flask import Flask, jsonify
import random

app = Flask(__name__)

# A small fake "database" of products.
PRODUCTS = [
    {"id": 1, "name": "Wireless Mouse", "price": 19.99, "category": "Electronics"},
    {"id": 2, "name": "Mechanical Keyboard", "price": 89.50, "category": "Electronics"},
    {"id": 3, "name": "Standing Desk", "price": 249.00, "category": "Furniture"},
    {"id": 4, "name": "Office Chair", "price": 175.25, "category": "Furniture"},
    {"id": 5, "name": "USB-C Hub", "price": 34.99, "category": "Electronics"},
    {"id": 6, "name": "Desk Lamp", "price": 22.40, "category": "Furniture"},
    {"id": 7, "name": "Noise-Cancelling Headphones", "price": 199.99, "category": "Electronics"},
    {"id": 8, "name": "Bookshelf", "price": 89.00, "category": "Furniture"},
    # Product 9 deliberately has NO review page, to test error handling
    {"id": 9, "name": "Mystery Gadget", "price": 9.99, "category": "Electronics"},
    {"id": 10, "name": "Ergonomic Footrest", "price": 29.99, "category": "Furniture"},
]

# Deterministic fake reviews so results are reproducible while you test.
random.seed(42)
REVIEWS = {
    p["id"]: {
        "avg_score": round(random.uniform(2.5, 5.0), 1),
        "review_count": random.randint(0, 500),
    }
    for p in PRODUCTS if p["id"] != 9 # product 9 has no reviews on purpose
}

@app.route("/api/products", methods=["GET"])
def get_all_products():
    """Returns the full product list as JSON (simulates the real API)."""
    return jsonify(PRODUCTS)

@app.route("/api/products/<int:product_id>", methods=["GET"])
def get_product(product_id):
    """Returns a single product, or a 404 if it doesn't exist."""
    for p in PRODUCTS:
        if p["id"] == product_id:
            return jsonify(p)
    return jsonify({"error": "Product not found"}), 404

@app.route("/reviews/<int:product_id>", methods=["GET"])
def get_review_page(product_id):
    """
    Returns a small HTML page containing review info, mimicking a real
    website that scraper.py will parse with BeautifulSoup.
    """
    if product_id not in REVIEWS:
        # No review page exists for this product -> simulate a 404
        return "<html><body><h1>404 - Not Found</h1></body></html>", 404

    data = REVIEWS[product_id]
    html = f"""
<html>
  <body>
    <div class="review-summary">
      <span class="average-score">{data['avg_score']}</span>
      <span class="review-count">{data['review_count']} reviews</span>
    </div>
  </body>
</html>
"""
    return html

if __name__ == "__main__":
    print("Mock server running at http://127.0.0.1:5000")
    print(" Product list: http://127.0.0.1:5000/api/products")
    print(" Example review: http://127.0.0.1:5000/reviews/1")
    app.run(port=5000, debug=False)

```

5. api_fetcher.py — API Interaction

[Intermediate] Key ideas: making HTTP requests, handling network failures gracefully instead of crashing, and caching.

```
"""
api_fetcher.py
-----
Responsible for ONE job: talking to the product API and turning its
JSON responses into plain Python dictionaries the rest of the app can use.

Key concepts demonstrated here:
- requests library for HTTP calls
- JSON parsing
- Error handling (timeouts, bad status codes, connection errors)
- functools.lru_cache to avoid re-fetching the same product twice
"""

import requests
from functools import lru_cache
from config import API_BASE_URL, REQUEST_TIMEOUT

def fetch_all_products():
    """
    Fetches the full product list from the API.
    Returns a list of dicts, e.g. [{"id": 1, "name": "...", "price": ...}, ...]
    Returns an empty list if the request fails, so callers don't crash.
    """
    try:
        response = requests.get(API_BASE_URL, timeout=REQUEST_TIMEOUT)
        response.raise_for_status() # raises an exception for 4xx/5xx codes
        return response.json()
    except requests.exceptions.Timeout:
        print(f"[api_fetcher] Timed out after {REQUEST_TIMEOUT}s contacting {API_BASE_URL}")
        return []
    except requests.exceptions.ConnectionError:
        print(f"[api_fetcher] Could not connect to {API_BASE_URL}. Is the mock server running?")
        return []
    except requests.exceptions.HTTPError as e:
        print(f"[api_fetcher] API returned an error status: {e}")
        return []
    except ValueError:
        # response.json() failed to parse -> not valid JSON
        print("[api_fetcher] API response was not valid JSON.")
        return []

@lru_cache(maxsize=128)
def fetch_product(product_id):
    """
    Fetches a SINGLE product by ID.

    @lru_cache means: if this function is called twice with the same
    product_id, the second call returns the cached result instantly
    instead of hitting the network again. This directly satisfies the
    assignment's "cache API responses for products requested multiple
    times" requirement.

    NOTE: lru_cache requires arguments to be hashable, which a plain
    int (product_id) already is, so no extra work is needed here.
    """
    url = f"{API_BASE_URL}/{product_id}"
    try:
        response = requests.get(url, timeout=REQUEST_TIMEOUT)
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"[api_fetcher] Failed to fetch product {product_id}: {e}")
        return None
```

Why try/except with specific exception types instead of one catch-all? Catching `requests.exceptions.Timeout` separately from `ConnectionError` lets you print a message that actually tells you what went wrong, which matters a lot when debugging a real network integration.

Why @lru_cache ? It's a **decorator** — a function that wraps another function to add behavior without changing its code. `lru_cache` specifically remembers previous return values keyed by the arguments passed in. This is a topic you'll hit after functions and before classes — worth bookmarking.

6. scraper.py — Web Scraping

[Intermediate] Downloads raw HTML and picks specific pieces out of it using BeautifulSoup.

```

"""
scraper.py
-----
Responsible for ONE job: given a product ID, fetch its review page (HTML)
and extract the average score and review count using BeautifulSoup.

Key concepts demonstrated here:
- requests to download raw HTML
- BeautifulSoup to parse HTML and find specific elements by CSS class
- Graceful handling of missing pages (product has no reviews / 404)
"""

import requests
from bs4 import BeautifulSoup
from config import SCRAPER_BASE_URL, REQUEST_TIMEOUT

def scrape_review_page(product_id):
    """
    Scrapes the review page for a single product.

    Returns a dict: {"avg_score": float, "review_count": int}
    Returns None if the page doesn't exist or has no review data,
    so the caller can filter these products out later.
    """
    url = f"{SCRAPER_BASE_URL}/{product_id}"

    try:
        response = requests.get(url, timeout=REQUEST_TIMEOUT)

        if response.status_code == 404:
            # This product simply has no review page - not a crash,
            # just a normal "no data" case.
            return None

        response.raise_for_status()

    except requests.exceptions.RequestException as e:
        print(f"[scraper] Failed to reach review page for product {product_id}: {e}")
        return None

    # "html.parser" is Python's built-in parser - no extra install needed.
    soup = BeautifulSoup(response.text, "html.parser")

    score_tag = soup.find(class_="average-score")
    count_tag = soup.find(class_="review-count")

    if score_tag is None or count_tag is None:
        # The page loaded, but didn't have the elements we expected.
        return None

    try:
        avg_score = float(score_tag.text.strip())
        # count_tag.text looks like "123 reviews" -> pull out the number
        review_count = int(count_tag.text.strip().split()[0])
    except (ValueError, IndexError):
        print(f"[scraper] Could not parse review numbers for product {product_id}")
        return None

    return {"avg_score": avg_score, "review_count": review_count}

```

How `soup.find(class_="average-score")` **works:** BeautifulSoup turns the HTML text into a searchable tree. `find` looks for the first tag with a matching CSS class, the same way your browser's inspector highlights elements. `class_` has a trailing underscore because `class` alone is a reserved Python keyword.

7. `data_models.py` — The `Product` Class (`__slots__`)

[Intermediate — right after you learn classes] This is the custom data structure the assignment requires.

```

"""
data_models.py
-----
Defines the Product class - the custom data structure this project
uses to represent one product with all its combined data (from the
API AND the scraper).

Key concept: __slots__

Normally, every Python object stores its attributes in a hidden
dictionary (__dict__), which is flexible but uses extra memory per
object. If you're creating THOUSANDS of Product objects (like a real
market-research firm would), that overhead adds up.

__slots__ tells Python: "this class will ONLY ever have these exact
attributes - don't bother creating a __dict__ for it." This:
- reduces memory usage per instance (often 30-50% less)
- prevents typos like product.pric = 10 (raises AttributeError
  instead of silently creating a new, wrong attribute)

The trade-off: you can't add new attributes on the fly, which is fine
here because we know exactly what a Product looks like.
"""

class Product:
    __slots__ = ("id", "name", "price", "category", "avg_score", "review_count")

    def __init__(self, id, name, price, category, avg_score=None, review_count=None):
        self.id = id
        self.name = name
        self.price = price
        self.category = category
        self.avg_score = avg_score # None until scraper.py fills it in
        self.review_count = review_count # None until scraper.py fills it in

    def has_reviews(self):
        """True if this product has at least one review."""
        return self.review_count is not None and self.review_count > 0

    def __repr__(self):
        return (f"Product(id={self.id}, name={self.name!r}, price={self.price}, "
            f"category={self.category!r}, avg_score={self.avg_score}, "
            f"review_count={self.review_count})")

    def to_dict(self):
        """Convert back to a plain dict - useful for JSON output."""
        return {
            "id": self.id,
            "name": self.name,
            "price": self.price,
            "category": self.category,
            "avg_score": self.avg_score,
            "review_count": self.review_count,
        }

```

A dictionary connection, since that's where you are right now: `to_dict()` converts a `Product` object back into a plain dictionary — the same kind of structure you're currently working with (`{"id": 1, "name": "Mouse", ...}`). Classes are, in a sense, dictionaries with fixed, named fields and behavior (methods) attached. Understanding dictionaries well is exactly what makes classes click later.

8. analyzer.py — Functional Programming (`map`, `filter`, `reduce`)

[Intermediate/Advanced] These three functions are a different style of processing collections than a `for` loop.

```

"""
analyzer.py
-----
Takes a list of Product objects and analyzes them using functional
programming tools: map, filter, and functools.reduce.

WHY these instead of a for-loop? A for-loop works fine too – but the
assignment specifically wants you to practice the functional style,
which is common in data-processing code because it expresses
"transform this whole collection" more declaratively than a loop with
manual accumulator variables.
"""

from functools import reduce

def filter_products_with_reviews(products):
    """
    Keep only products that HAVE reviews.
    filter(function, iterable) keeps items where function(item) is True.
    """
    return list(filter(lambda p: p.has_reviews(), products))

def average_price_per_category(products):
    """
    Returns a dict like {"Electronics": 85.4, "Furniture": 130.2}

    Approach:
    1. Get the unique categories (a set comprehension).
    2. For each category, use `filter` + `map` to get just the prices
       of products in that category.
    3. Use `reduce` to sum those prices, then divide by count.
    """
    categories = {p.category for p in products}
    result = {}

    for category in categories:
        # filter: only products in this category
        in_category = filter(lambda p, c=category: p.category == c, products)
        # map: pull out just the price from each product
        prices = list(map(lambda p: p.price, in_category))

        if not prices:
            continue

        # reduce: combine all prices into a single sum
        total = reduce(lambda acc, price: acc + price, prices, 0)
        result[category] = round(total / len(prices), 2)

    return result

def topRated_products(products, n=5):
    """
    Returns the top `n` products sorted by avg_score, highest first.
    Products with no score (None) are excluded first via filter.
    """
    scored = filter(lambda p: p.avg_score is not None, products)
    return sorted(scored, key=lambda p: p.avg_score, reverse=True)[:n]

```

Quick primer, since these are new: - `filter(function, list)` — keeps only items where `function(item)` returns `True`. Equivalent to `[x for x in list if function(x)]`. - `map(function, list)` — applies `function` to every item, returning a new sequence of results. Equivalent to `[function(x) for x in list]`. - `reduce(function, list, start)` — repeatedly applies `function(accumulator, item)` to collapse a list into ONE value (here, summing prices). - `lambda arg: expression` — a tiny, unnamed function written inline, used because these functions are small and only used once.

If list comprehensions feel more natural to you right now, that's completely fine — they do the same job and you'll meet `map / filter / reduce` again naturally once you're comfortable with functions.

9. `reporter.py` — Output Generation

[Beginner/Intermediate — mostly things you already know]

```

"""
reporter.py
-----
Takes the analysis results and presents them:
1. Printed nicely to the console.
2. Written to a JSON file (report.json) for machine-readable output.
"""

import json
from config import REPORT_OUTPUT_FILE

def print_report(top_products, avg_price_per_category, total_analyzed):
    print("\n" + "=" * 50)
    print("PRODUCT SENTIMENT & AVAILABILITY REPORT")
    print("=" * 50)

    print(f"\nTotal products analyzed: {total_analyzed}")

    print("\nAverage price per category:")
    for category, avg_price in avg_price_per_category.items():
        print(f"  {category:<15} ${avg_price:>8.2f}")

    print("\nTop rated products:")
    for i, product in enumerate(top_products, start=1):
        print(f"  {i}. {product.name:<30} score={product.avg_score} "
              f"({product.review_count} reviews)")

    print("\n" + "=" * 50 + "\n")

def write_json_report(top_products, avg_price_per_category, total_analyzed, filename=None):
    filename = filename or REPORT_OUTPUT_FILE

    report_data = {
        "total_products_analyzed": total_analyzed,
        "average_price_per_category": avg_price_per_category,
        "top_rated_products": [p.to_dict() for p in top_products],
    }

    with open(filename, "w") as f:
        json.dump(report_data, f, indent=2)

    print(f"[reporter] Report written to {filename}")

```

10. `main.py` — Orchestration & Two Concurrency Modes

[Advanced] This is the most conceptually demanding file. Take it slowly.

Why concurrency matters here

If we scraped each product's review page one after another in a plain loop, total time $\approx$ (time per request) $\times$ (number of products). Network requests spend most of their time **waiting**, not computing. Concurrency lets many requests be "in flight" at once.

- **Threads (`ThreadPoolExecutor`)** — run several threads, each doing a normal blocking `requests.get()` . Simple, and a great fit for this kind of I/O-bound work.
- **Asyncio (`asyncio` + `aiohttp`)** — runs on ONE thread but uses `await` to pause a task while it waits on the network, letting another task run in the meantime. More scalable, but requires `async def / await` syntax and a different HTTP library, since plain `requests` doesn't support it.

```

"""
main.py
-----
The entry point that ties everything together. Run it with either:

    python main.py --mode threads
    python main.py --mode async

Both modes do the SAME job (fetch all products, scrape all their
review pages, analyze, report) but use different concurrency
strategies, so you can compare their performance.
"""

import argparse
import asyncio
import cProfile

```

```

import pstats
import io

from concurrent.futures import ThreadPoolExecutor

from api_fetcher import fetch_all_products
from scraper import scrape_review_page
from data_models import Product
from analyzer import filter_products_with_reviews, average_price_per_category, top_rated_products
from reporter import print_report, write_json_report
from config import SCRAPER_BASE_URL, REQUEST_TIMEOUT

# -----
# MODE 1: ThreadPoolExecutor
# -----
def run_with_threads(raw_products):
    """
    Scrapes every product's review page concurrently using a pool of
    worker threads.

    ThreadPoolExecutor.map() applies scrape_review_page to every
    product id, distributing the calls across `max_workers` threads,
    and returns the results IN ORDER (matching the input order).
    """
    product_ids = [p["id"] for p in raw_products]

    with ThreadPoolExecutor(max_workers=8) as executor:
        review_results = list(executor.map(scrape_review_page, product_ids))

    return _build_products(raw_products, review_results)

# -----
# MODE 2: asyncio + aiohttp
# -----
async def _scrape_one_async(session, product_id):
    """Async version of scrape_review_page, using aiohttp instead of requests."""
    import aiohttp # imported here so `--mode threads` works even without aiohttp installed
    url = f"{SCRAPER_BASE_URL}/{product_id}"
    try:
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=REQUEST_TIMEOUT)) as resp:
            if resp.status == 404:
                return None
            resp.raise_for_status()
            html = await resp.text()
    except (aiohttp.ClientError, asyncio.TimeoutError):
        return None

    # Re-use the same parsing logic as scraper.py by parsing here with
    # BeautifulSoup (kept local to avoid importing requests-based code
    # into the async path).
    from bs4 import BeautifulSoup
    soup = BeautifulSoup(html, "html.parser")
    score_tag = soup.find(class_="average-score")
    count_tag = soup.find(class_="review-count")
    if score_tag is None or count_tag is None:
        return None

    try:
        return {
            "avg_score": float(score_tag.text.strip()),
            "review_count": int(count_tag.text.strip().split()[0]),
        }
    except (ValueError, IndexError):
        return None

async def _run_with_async_inner(raw_products):
    import aiohttp
    product_ids = [p["id"] for p in raw_products]

    async with aiohttp.ClientSession() as session:
        # asyncio.gather runs all these coroutines concurrently and
        # waits for all of them to finish, returning results in order.
        tasks = [_scrape_one_async(session, pid) for pid in product_ids]
        review_results = await asyncio.gather(*tasks)

    return _build_products(raw_products, review_results)

def run_with_async(raw_products):
    """Synchronous wrapper so main() can call this the same way as run_with_threads."""
    return asyncio.run(_run_with_async_inner(raw_products))

# -----
# Shared helper
# -----
def _build_products(raw_products, review_results):
    """Combines raw API product dicts with their scraped review data into Product objects."""

```

```

products = []
for raw, review in zip(raw_products, review_results):
    review = review or {}
    products.append(Product(
        id=raw["id"],
        name=raw["name"],
        price=raw["price"],
        category=raw["category"],
        avg_score=review.get("avg_score"),
        review_count=review.get("review_count"),
    ))
return products

# -----
# Orchestration
# -----
def run_pipeline(mode):
    raw_products = fetch_all_products()
    if not raw_products:
        print("No products fetched - is mock_server.py running?")
        return

    if mode == "threads":
        products = run_with_threads(raw_products)
    else:
        products = run_with_async(raw_products)

    with_reviews = filter_products_with_reviews(products)
    avg_prices = average_price_per_category(products)
    top5 = top_rated_products(with_reviews, n=5)

    print_report(top5, avg_prices, total_analyzed=len(products))
    write_json_report(top5, avg_prices, total_analyzed=len(products))

def main():
    parser = argparse.ArgumentParser(description="Multi-threaded / async data aggregator")
    parser.add_argument("--mode", choices=["threads", "async"], default="threads",
                        help="Concurrency strategy to use")
    parser.add_argument("--profile", action="store_true",
                        help="Run with cProfile and print the top time-consuming calls")
    args = parser.parse_args()

    if args.profile:
        profiler = cProfile.Profile()
        profiler.enable()
        run_pipeline(args.mode)
        profiler.disable()

        stats_stream = io.StringIO()
        stats = pstats.Stats(profiler, stream=stats_stream).sort_stats("cumulative")
        stats.print_stats(15) # top 15 slowest calls
        print("\n--- cProfile output (top 15 by cumulative time) ---")
        print(stats_stream.getvalue())
    else:
        run_pipeline(args.mode)

if __name__ == "__main__":
    main()

```

On `--profile` and `cProfile`: profiling means measuring exactly which function calls take the most time.

`cProfile.Profile()` records every function call while your code runs; `pstats.Stats(...).sort_stats("cumulative")` then sorts that data so the slowest parts appear first. This is how professional developers find real bottlenecks instead of guessing.

11. test_project.py — Unit Tests

[Advanced] Includes 7 tests, including one that uses `unittest.mock` to fake a network call so the test runs instantly and never touches the real network.

```

"""
test_project.py
-----
Unit tests for the core logic. Run with:

    pytest test_project.py -v

Includes at least one test that uses unittest.mock to patch a network
call, as required – see test_fetch_product_uses_mocked_network.
"""

```

```

from unittest.mock import patch, Mock

from data_models import Product
from analyzer import filter_products_with_reviews, average_price_per_category, topRatedProducts
import api_fetcher

def make_product(id, name, price, category, avg_score=None, review_count=None):
    return Product(id, name, price, category, avg_score, review_count)

# -----
# data_models tests
# -----
def test_product_has_reviews_true():
    p = make_product(1, "Mouse", 19.99, "Electronics", avg_score=4.5, review_count=10)
    assert p.has_reviews() is True

def test_product_has_reviews_false_when_zero():
    p = make_product(1, "Mouse", 19.99, "Electronics", avg_score=None, review_count=0)
    assert p.has_reviews() is False

def test_product_slots_prevents_new_attribute():
    """__slots__ should stop you from adding attributes that don't exist."""
    p = make_product(1, "Mouse", 19.99, "Electronics")
    try:
        p.some_new_field = "oops"
        assert False, "Expected AttributeError due to __slots__"
    except AttributeError:
        pass

# -----
# analyzer tests
# -----
def test_filter_products_with_reviews():
    products = [
        make_product(1, "A", 10, "Cat1", avg_score=4.0, review_count=5),
        make_product(2, "B", 20, "Cat1", avg_score=None, review_count=0),
    ]
    result = filter_products_with_reviews(products)
    assert len(result) == 1
    assert result[0].id == 1

def test_average_price_per_category():
    products = [
        make_product(1, "A", 10, "Electronics"),
        make_product(2, "B", 30, "Electronics"),
        make_product(3, "C", 100, "Furniture"),
    ]
    result = average_price_per_category(products)
    assert result["Electronics"] == 20.0
    assert result["Furniture"] == 100.0

def test_top_rated_products_orders_correctly():
    products = [
        make_product(1, "A", 10, "Cat", avg_score=3.0, review_count=5),
        make_product(2, "B", 10, "Cat", avg_score=4.9, review_count=5),
        make_product(3, "C", 10, "Cat", avg_score=4.0, review_count=5),
    ]
    top = top_rated_products(products, n=2)
    assert [p.id for p in top] == [2, 3]

# -----
# Mocked network test (required by the assignment)
# -----
def test_fetch_product_uses_mocked_network():
    """
    Uses unittest.mock.patch to replace requests.get with a fake object,
    so this test runs instantly and never touches the real network.
    """
    api_fetcher.fetch_product.cache_clear() # reset lru_cache between test runs

    fake_response = Mock()
    fake_response.json.return_value = {"id": 1, "name": "Mouse", "price": 19.99}
    fake_response.raise_for_status.return_value = None

    with patch("api_fetcher.requests.get", return_value=fake_response) as mock_get:
        result = api_fetcher.fetch_product(1)

        mock_get.assert_called_once()
        assert result == {"id": 1, "name": "Mouse", "price": 19.99}

```

Why `unittest.mock.patch`? Real unit tests shouldn't depend on a real network connection, a running server, or the

current state of some external website — they should run identically every time, instantly, and in isolation.

`patch("api_fetcher.requests.get", ...)` temporarily replaces the real `requests.get` function with a fake (“mock”) object that returns whatever data you tell it to, for the duration of the `with` block only.

Verified: all 7 tests pass when run in this environment.

12. Verified Sample Output

This is real output from actually running `python main.py --mode threads --profile` against the mock server described above.

```
=====
PRODUCT SENTIMENT & AVAILABILITY REPORT
=====

Total products analyzed: 10

Average price per category:
  Furniture      $ 113.13
  Electronics    $  70.89

Top rated products:
  1. USB-C Hub          score=4.7 (44 reviews)
  2. Mechanical Keyboard score=4.4 (125 reviews)
  3. Wireless Mouse     score=4.1 (12 reviews)
  4. Desk Lamp          score=4.0 (16 reviews)
  5. Standing Desk      score=3.1 (377 reviews)
=====

[reporter] Report written to report.json
```

Note that 10 products were fetched, but “Mystery Gadget” (product 9) never appears in the top-rated list — it has no review page at all, so `scraper.py` correctly returned `None` for it and `analyzer.py` filtered it out. This confirms the error-handling path works.

Resulting `report.json`

```
{
  "total_products_analyzed": 10,
  "average_price_per_category": {
    "Furniture": 113.13,
    "Electronics": 70.89
  },
  "top_rated_products": [
    { "id": 5, "name": "USB-C Hub", "price": 34.99, "category": "Electronics", "avg_score": 4.7, "review_count": 44 },
    { "id": 2, "name": "Mechanical Keyboard", "price": 89.5, "category": "Electronics", "avg_score": 4.4, "review_count": 125 },
    { "id": 1, "name": "Wireless Mouse", "price": 19.99, "category": "Electronics", "avg_score": 4.1, "review_count": 12 },
    { "id": 6, "name": "Desk Lamp", "price": 22.4, "category": "Furniture", "avg_score": 4.0, "review_count": 16 },
    { "id": 3, "name": "Standing Desk", "price": 249.0, "category": "Furniture", "avg_score": 3.1, "review_count": 377 }
  ]
}
```

Test run results

```
PASS: test_average_price_per_category
PASS: test_fetch_product_uses_mocked_network
PASS: test_filter_products_with_reviews
PASS: test_product_has_reviews_false_when_zero
PASS: test_product_has_reviews_true
PASS: test_product_slots_prevents_new_attribute
PASS: test_top_rated_products_orders_correctly

7 passed, 0 failed
```

13. Threads vs. Asyncio — What the Report.md

Deliverable Should Say

The assignment's own `REPORT.md` deliverable (max 2 pages) asks you to compare `ThreadPoolExecutor` vs `asyncio`, interpret `cProfile` output, and justify your data-structure choices. Here's the substance to put in your own words:

Performance comparison. For a small number of requests (like our 10 mock products) on localhost, both modes finish in well under a second, and the difference is negligible — network latency this low doesn't stress either approach. The real difference shows up at scale: with hundreds or thousands of slow, real-world HTTP requests, `asyncio` typically pulls ahead because a single thread juggling many "waiting" coroutines has less overhead than the OS context-switching between many actual threads. `ThreadPoolExecutor` is usually simpler to write and reason about, so it's a very reasonable choice unless you specifically need to scale to very high concurrency.

cProfile interpretation. The profiler output above shows most cumulative time spent inside `threading.py:wait` and the underlying socket/connection calls — confirming the program is *I/O-bound* (waiting on the network) rather than *CPU-bound* (doing heavy computation). That's exactly the situation where concurrency helps, and exactly why a plain sequential loop would be the slowest option of all.

Why `__slots__` and a custom `Product` class. A plain dictionary per product would also work, but a class gives you: named, fixed fields (typos become errors instead of silent bugs), methods like `has_reviews()` living next to the data they operate on, and — via `__slots__` — lower memory use per instance, which matters if this tool ever processes thousands of products instead of ten.

14. What to Study Next (In Order)

Since you're currently on dictionaries, here's a realistic path from where you are to being able to write this project yourself:

1. **Dictionaries & lists** (current) → comprehensions
2. **Functions**, default arguments, `*args / **kwargs`
3. **Exceptions** (`try / except`) — used constantly in this project
4. **Classes & OOP** — needed for `data_models.py`
5. **File I/O & JSON** — needed for `reporter.py`
6. **The `requests` library & HTTP basics** — needed for `api_fetcher.py`
7. **HTML basics & BeautifulSoup** — needed for `scraper.py`
8. **Decorators** (`@lru_cache`) — a nice "aha" moment once functions feel natural
9. `map / filter / reduce / lambda` — an alternate style to loops
10. **Threading, then `asyncio`** — genuinely one of the harder topics in Python; most courses put this near the end for a reason
11. **Unit testing with `pytest` and `unittest.mock`**

You don't need all of this to finish your current dictionary work — just keep this as a reference for what's coming.